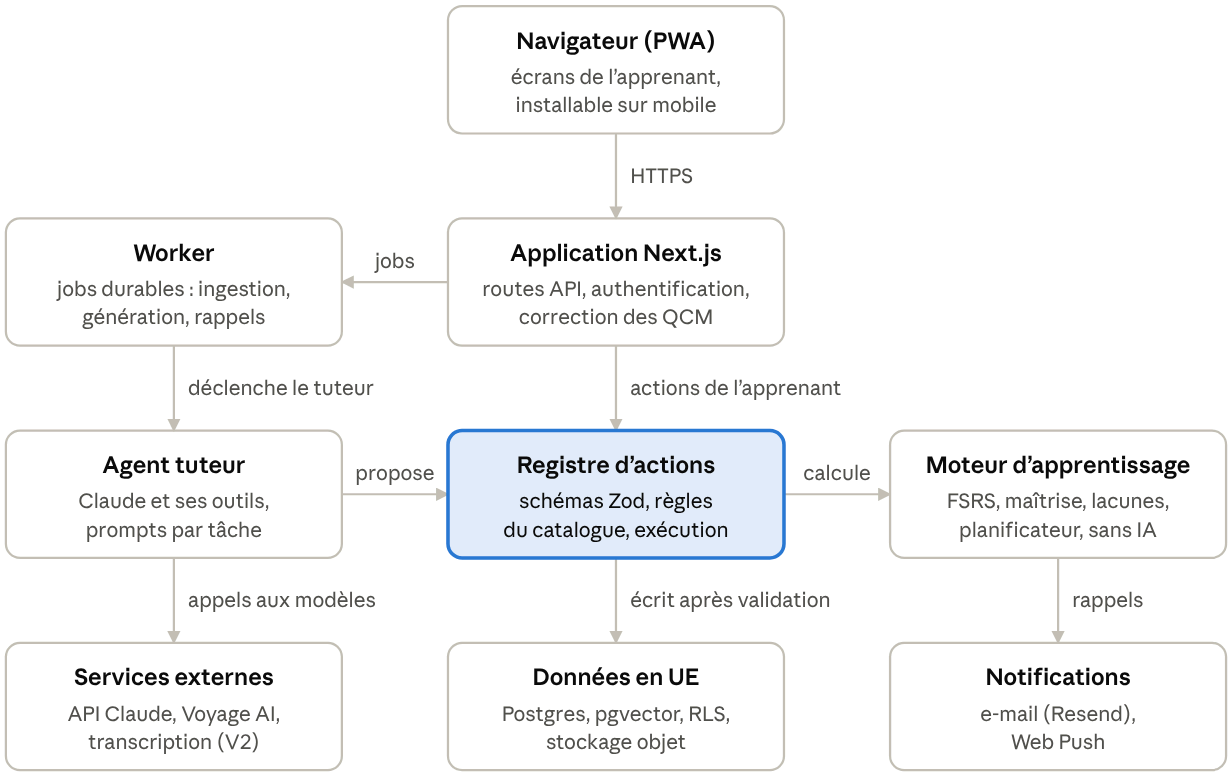
Architecture technique

Recommandation : une application web en TypeScript où le tuteur Claude propose des actions, et où le code les valide puis les exécute. Le moteur d'apprentissage reste déterministe et n'appelle jamais l'IA.

Vue d'ensemble

Le tuteur propose, l'application valide et exécute



architecture · 9 composants, le registre d'actions comme point de passage

Toute écriture passe par le registre d'actions, qu'elle vienne de l'apprenant ou du tuteur. Le worker déclenche le tuteur sur les événements ; le moteur d'apprentissage calcule tout ce qui doit être exact.

Stack recommandée

TypeScript de bout en bout, sur des services managés hébergés en UE : un seul langage pour le web, le worker et le moteur, et peu d'infrastructure à opérer.

Couche	Choix	Pourquoi	Alternative
Langage	TypeScript	Types partagés entre écrans, API, worker et schémas de briques	—
Application web	Next.js (App Router) + Tailwind CSS	Rendu serveur, routes API et PWA dans un seul projet	Remix, SvelteKit
Base de données	Supabase Cloud en région UE (Postgres + pgvector), auto-hébergeable plus tard	Données relationnelles et recherche vectorielle au même endroit	Neon + pgvector
Accès aux données	Drizzle ORM	Schéma typé, migrations SQL lisibles	Prisma
Isolation des données	Row Level Security de Postgres	Chaque apprenant ne voit que ses données, même en cas de bug applicatif	Filtrage applicatif seul
Fichiers	Stockage compatible S3 (Supabase Storage)	PDF, audio et vidéo hors base, servis par URL signées	Cloudflare R2
Tâches longues	Jobs durables (Inngest)	Ingestion et génération de cours sur plusieurs minutes, avec reprise	Trigger.dev
Authentification	Supabase Auth	Cohérente avec la base et la Row Level Security	Auth.js
Notifications	Resend (e-mail) + Web Push (VAPID)	Rappels sans application mobile, conforme au choix « web d'abord »	Postmark
Hébergement	Vercel, fonctions en région UE	Déploiement simple de Next.js, aperçu par branche	Render, Fly.io

Couche	Choix	Pourquoi	Alternative
Supervision	Sentry + journal des appels IA	Erreurs, latence et coût de chaque appel	OpenTelemetry
Couche modèles	AI SDK de Vercel	Un même code pour Claude, OpenAI, DeepSeek, Mistral et tout point d'accès compatible OpenAI	LiteLLM

Supabase regroupe base, authentification et stockage : un seul fournisseur en V1, remplaçable brique par brique ensuite.

Backend et API

Le backend est intégré à l'application Next.js : il n'y a pas de service d'API séparé en V1.

Couche	Rôle	Où ça tourne
Navigateur	Interface React, appels à l'API	Chez l'apprenant
API (backend Next.js)	Routes `/api/v1/…` et actions serveur : vérification de l'utilisateur, règles métier, appels au tuteur et au moteur d'apprentissage, validation des briques, écritures en base	Serveur (fonctions Vercel)
Worker (Inngest)	Traitements longs découpés en étapes : ingestion, génération de cours, rappels	Serveur, déclenché par des événements
Supabase	Base Postgres avec Row Level Security, authentification, fichiers	Supabase Cloud en région UE

- L'API est une API REST versionnée, décrite par les schémas Zod : une future application mobile ou un client externe pourra l'utiliser.
- Les routes ne contiennent pas de logique métier : elles vérifient l'utilisateur et appellent les paquets `tutor`, `bricks`, `learning-engine` et `llm`.
- Les appels aux modèles d'IA et toutes les clés restent côté serveur ; le navigateur n'appelle jamais un modèle directement.

- Sortir l'API dans un service séparé (Fastify, Hono ou NestJS) reste possible sans réécriture, si une application mobile native, une API publique ou l'auto-hébergement l'exigent.

Intelligence artificielle

Le tuteur s'appuie sur l'API Claude d'Anthropic, avec un modèle choisi par tâche : le plus capable pour concevoir, le plus rapide pour échanger.

Tâche	Modèle	Pourquoi
Concevoir le programme, rédiger le cours, analyser les lacunes	claude-opus-5-5	Raisonnement long, synthèse de sources hétérogènes ; tester Sonnet pour la rédaction du cours, moins cher si la qualité tient
Tuteur en chat, génération de questions et de jeux, correction des réponses ouvertes	claude-sonnet-5-5	Équilibre qualité, latence et coût pour les tâches fréquentes
Classification, découpage, étiquetage des apports, variantes simples de questions (texte à trous, cartes de rappel)	claude-haiku-5-5	Tâches simples et volumineuses, coût très bas

Les identifiants de modèle sont regroupés dans un seul fichier de configuration, pour en changer sans toucher au code.

Fonctionnalités de l'API utilisées

- **Outils** : chaque action du catalogue est un outil décrit par un schéma JSON, en mode strict pour garantir des arguments conformes. Le SDK TypeScript les déclare directement avec Zod.
- **Choix d'outil automatique** : ces modèles n'acceptent pas le choix forcé d'un outil ; le prompt indique quel outil appeler.
- **Validation** : l'application revalide chaque appel ; une erreur revient au modèle comme résultat d'outil en erreur, qu'il corrige.
- **Sorties structurées** : pour une génération sans outil, une banque de questions par exemple, la réponse est contrainte par un schéma.

- **Recherche web** : outil côté serveur d'Anthropic, réservé à la conception du programme et aux suggestions de ressources.
- **PDF** : lus nativement par le modèle pour rédiger le cours, envoyés une fois via l'API Files puis référencés ; le tuteur en chat ne reçoit que les passages utiles.
- **Citations** : activées sur les documents pour relier chaque passage du cours à sa source. Elles ne se combinent pas avec les sorties structurées : le cours est rédigé en texte cité, puis découpé en briques.
- **Mise en cache** : le contexte stable d'une formation (instructions, outils, programme) est placé en tête des requêtes et mis en cache ; une lecture en cache coûte jusqu'à 20 fois moins cher qu'une entrée normale.
- **Traitement par lots** : les générations non urgentes, comme les banques de questions préparées la nuit, passent par l'API Batches, moins chère.
- **Réflexion adaptative et effort** : effort élevé pour concevoir le programme, faible pour le chat.
- **Repli en cas de refus** : le repli côté serveur est activé, pour qu'une requête refusée à tort bascule sur un autre modèle.

Autres services IA

- **Embeddings** : Voyage AI, pour retrouver dans les apports les passages utiles à une question (RAG), stockés dans pgvector. Modèle voyage-4-lite : 200 millions de tokens offerts par compte, puis 0,02 $ par million.
- **Transcription audio et vidéo (V2)** : une API de reconnaissance vocale, par exemple Deepgram ou AssemblyAI.

Modèles configurables par l'utilisateur

Chaque utilisateur peut remplacer Claude, tâche par tâche, par le modèle de son choix et sa propre clé d'API. Claude reste le modèle par défaut.

Comment ça marche

- Une couche unique, l'AI SDK de Vercel, parle à tous les fournisseurs : Anthropic, OpenAI, DeepSeek, Mistral, tout point d'accès compatible OpenAI, et des modèles locaux.
- Pour chaque tâche, l'utilisateur choisit un fournisseur et un modèle, puis renseigne sa clé ou l'adresse de son point d'accès.

- Le code ne connaît que des tâches : le paquet `llm` résout le modèle à appeler pour cet utilisateur et cette tâche.

Tâche	Modèle par défaut	Capacités exigées du modèle
Conception du programme	Claude Opus	Appel d'outils, sortie structurée, recherche web
Rédaction du cours	Claude Opus	Sortie structurée, références aux passages des apports
Analyse des lacunes	Claude Opus	Sortie structurée
Tuteur, questions, jeux, correction	Claude Sonnet	Appel d'outils, sortie structurée
Tâches légères (classification, découpage)	Claude Haiku	Sortie structurée

Ce qui remplace les fonctions propres à Claude

- Recherche web : une API de recherche fournie par la plateforme.
- Citations : chaque passage du cours renvoie à un extrait des apports retrouvé par la recherche vectorielle, vérifié par le code.
- Lecture des PDF : le texte extrait à l'ingestion.
- Mise en cache : celle du fournisseur choisi, quand il en propose une.

Garde-fous

- Qualification : à l'enregistrement, un test de connexion puis une batterie d'essais (appel d'outils, format de sortie, une brique de chaque type). Un modèle qui échoue n'est pas activé.
- Clés : chiffrées au repos, jamais renvoyées au navigateur, affichées masquées, supprimables à tout moment.
- Données : un avertissement s'affiche si le fournisseur choisi traite les données hors de l'Union européenne.
- Échecs répétés : la tâche s'arrête et l'utilisateur est prévenu ; il choisit un autre modèle de sa clé ou l'IA fournie, payante.
- Le moteur d'apprentissage (FSRS, maîtrise, lacunes, planning) ne dépend d'aucun modèle : il fonctionne à l'identique.

Organisation du code

Un monorepo pnpm et Turborepo : deux applications et cinq paquets métier. Le moteur d'apprentissage n'appelle jamais l'IA, ce qui le rend testable et prévisible.

```
tuteur/
├── apps/
|   ├── web/                  Next.js : écrans, routes API, PWA
|   └── worker/               jobs durables : ingestion, génération, rappels
├── packages/
|   ├── bricks/               un dossier par type de brique
|   |   ├── question-choix/   schema.ts (Zod) · view.tsx (React) · grade.ts
|   |   ├── texte-a-trous/
|   |   ├── …
|   |   └── registry.ts       registre des types de briques
|   ├── tutor/                agent tuteur
|   |   ├── prompts/          instructions par tâche
|   |   ├── tools/            une action du catalogue par fichier
|   |   ├── loop.ts           boucle agent et garde-fous
|   |   └── models.ts         tâches et capacités exigées
|   ├── learning-engine/      TypeScript pur, aucun appel IA
|   |   ├── fsrs/             répétition espacée par notion
|   |   ├── mastery/          niveau de maîtrise
|   |   ├── gaps/             détection et nature des lacunes
|   |   ├── selection/        tirage des tentatives et du quiz global
|   |   └── planner/          agenda, marges, options en cas de blocage
|   ├── llm/                  fournisseurs (AI SDK), modèle par utilisateur
et par tâche, clés chiffrées, qualification
|   ├── ingestion/            extraction PDF, découpage, embeddings
|   └── db/                   schéma Drizzle, migrations, politiques RLS
└── turbo.json
```

Ajouter un type de brique revient à créer un dossier dans `packages/bricks` et à l'enregistrer ; rien d'autre ne change.

Patterns

Un principe gouverne tous les autres : le modèle propose, le code valide et exécute.

Pattern	Rôle dans l'outil
Agent orchestrateur et registre d'outils	Le tuteur ne connaît que les actions du registre ; ajouter une action, c'est ajouter un outil
Le modèle propose, le code valide	Chaque sortie du tuteur passe par un schéma Zod et les règles du catalogue avant d'être enregistrée
Briques versionnées	Une modification crée une version ; les résultats restent liés à la version utilisée
Événements de domaine	QuizTerminé, ModuleValidé, ApportsTerminés, NotionDue déclenchent le tuteur via un job
Jobs idempotents et reprenables	Une génération de cours interrompue reprend à l'étape suivante, sans doublon
Séparation déterministe et génératif	Dates, scores, seuils et déblocages sont calculés par le code ; contenus et explications par le modèle
Contenus non fiables isolés	Les apports sont transmis au modèle comme données, jamais comme instructions
Le code avant l'IA	QCM, jeux, dates de révision, scores et déblocages ne font aucun appel à un modèle
Générer une fois, réutiliser	Cours stocké par version de module ; banque de questions par notion, préparée à l'avance et partagée
Budget par appel	Chaque appel déclare sa tâche ; le paquet llm applique le modèle, l'effort et le plafond, puis enregistre le coût

Exemple : fin d'un quiz de validation

1. L'application corrige les QCM et les jeux, enregistre la tentative et émet QuizTerminé.
2. Un job lance le tuteur, qui corrige les réponses ouvertes avec l'action « Évaluer une réponse ».
3. Le moteur d'apprentissage calcule le score, met à jour FSRS et la maîtrise, puis décide du déblocage.
4. Sous le seuil, le tuteur qualifie les lacunes et génère la remédiation.

5. Le planificateur réserve une marge et programme les rappels.

Algorithmes

Tous ces algorithmes vivent dans le moteur d'apprentissage, en TypeScript pur et testés unitairement ; aucun n'appelle l'IA.

Algorithme	Entrées	Règle	Sortie
Répétition espacée	Historique des réponses par notion	FSRS (bibliothèque ts-fsrs), suivi par notion ; note déduite de la justesse et du temps de réponse ; auto-évaluation pour la carte de rappel	Prochaine date de révision, probabilité de rappel
Niveau de maîtrise	Réponses à la notion	Moyenne des réussites pondérée par la récence	Score de 0 à 100 %
Détection de lacune	Maîtrise, historique des tentatives	Lacune si la maîtrise reste sous le seuil après une tentative, ou si la même erreur revient	Notions en lacune
Nature de la lacune	Erreurs et leur contexte	Confusion : le distracteur choisi représente une autre notion. Oubli : notion réussie puis échouée, signalée par FSRS. Incompréhension ou difficulté d'application : correction des réponses ouvertes par le tuteur	Nature de chaque lacune
Tirage d'une tentative	Notions du module, lacunes, questions déjà posées	Couvrir chaque notion, surpondérer les lacunes, exclure toute question déjà vue grâce à son empreinte	Questions tirées de la banque ; génération seulement si elle est épuisée

Algorithme	Entrées	Règle	Sortie
Banque de questions	Notions, formats de jeu, questions déjà vues	Plusieurs variantes par notion et par format, générées en lot (batch) et partagées entre apprenants d'un même programme ; régénérées quand un apprenant les a toutes vues	Variantes prêtes à tirer
Sélection du contexte	Tâche ou question, passages des apports	Recherche par embeddings, quelques passages les plus proches, dans un budget de tokens fixé par tâche	Contexte minimal envoyé au modèle
Garde-fou de coût	Estimation des tokens, dépense du jour	Estimation avant chaque appel ; au-delà du plafond, la tâche est reportée en lot ou refusée avec une alerte	Appel autorisé, différé ou refusé
Tirage du quiz global	Notions dues et lacunes de la formation	Notions dues selon FSRS, lacunes en tête, modules de la formation entrelacés, durée plafonnée	Session du jour
Planificateur	Programmes de toutes les formations, disponibilités, avancement	Glouton sous contraintes, sur un agenda commun à toutes les formations : place les activités dans les créneaux libres et réserve des marges. En cas de blocage, calcule trois options : décaler la fin, augmenter le volume quotidien, alléger le programme	Agenda et options proposées

Algorithme	Entrées	Règle	Sortie
Rétention	Résultats du quiz global	Part des notions réussies parmi celles revues 30 et 60 jours après la fin de leur module	Indicateur à 1 mois et à 2 mois

Chaque distracteur de QCM est généré avec la notion qu'il représente : c'est ce qui rend une confusion détectable. L'empreinte d'une question est un hachage de son énoncé normalisé et de sa bonne réponse.

Clés API et secrets

Les secrets restent côté serveur, dans le coffre de secrets de l'hébergeur ; le navigateur ne reçoit que des valeurs publiques par conception.

Variable	Service	Obligatoire en V1
ANTHROPIC_API_KEY	API Claude (Anthropic)	Oui
VOYAGE_API_KEY	Embeddings Voyage AI	Oui
DATABASE_URL	Postgres	Oui
SUPABASE_SERVICE_ROLE_KEY	Supabase côté serveur : auth, stockage, accès privilégié	Oui
NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase côté navigateur, protégé par la Row Level Security	Oui, valeurs publiques
RESEND_API_KEY	E-mails de rappel	Oui
VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY	Notifications Web Push	Oui, seule la clé publique va au navigateur

Variable	Service	Obligatoire en V1
INNGEST_EVENT_KEY, INNGEST_SIGNING_KEY	Jobs durables	Oui
AUTH_SECRET	Signature des sessions, si Auth.js remplace Supabase Auth	Non
SENTRY_DSN	Supervision des erreurs	Recommandé
Clé de transcription (Deepgram ou AssemblyAI)	Audio et vidéo	Non, V2
APP_ENCRYPTION_KEY	Chiffrement des clés d'API fournies par les utilisateurs	Oui
SEARCH_API_KEY	API de recherche web (par exemple Brave Search ou Tavily), pour les modèles sans recherche intégrée	Oui, dès qu'un utilisateur choisit un tel modèle

Une clé Anthropic par environnement (développement, préproduction, production), avec un plafond de dépense sur chacun.

Les clés fournies par les utilisateurs ne sont pas des variables d'environnement : elles sont stockées chiffrées, une par utilisateur et par fournisseur.

Sécurité, RGPD et coûts

Les apports des apprenants sont des données personnelles : ils restent en UE, isolés par apprenant, et ne servent jamais d'instructions au tuteur.

Sécurité et RGPD

- Base, fichiers et fonctions hébergés en UE ; contrat de sous-traitance signé avec chaque fournisseur.
- Row Level Security : chaque requête est filtrée par apprenant au niveau de la base.
- Chiffrement au repos et en transit ; fichiers servis par URL signées à durée courte.

- Export et suppression complète du compte en libre-service.
- Conditions de rétention et d'usage des données des fournisseurs IA vérifiées et documentées avant le lancement.
- Injection de prompt : les apports sont balisés comme données ; les actions du tuteur restent bornées par le registre et la validation.

Maîtrise des coûts

Poste	Tarif public	Usage
Claude Opus 5.5	4 $ en entrée, 20 $ en sortie, par million de tokens	Programme, rédaction du cours, analyse des lacunes
Claude Sonnet 5.5	2 $ / 10 $	Tuteur, questions, correction
Claude Haiku 5.5	0,10 $ / 0,50 $ (requêtes jusqu'à 100 000 tokens)	Tâches légères ; environ 40 fois moins cher qu'Opus
Voyage voyage-4-lite	200 millions de tokens offerts, puis 0,02 $ par million	Embeddings des apports ; coût négligeable

Tarifs relevés en octobre 2026 (Anthropic) et septembre 2026 (Voyage), à revérifier avant le lancement.

Leviers, du plus rentable au moins rentable :

1. **Pas d'IA quand le code suffit** : QCM et jeux corrigés par l'application, dates de révision calculées par FSRS.
2. **Générer une fois, réutiliser** : un cours par module ; une banque de variantes de questions par notion, régénérée seulement quand elle est épuisée et partagée entre apprenants d'un même programme tant qu'ils ne les ont pas vues.
3. **Le bon modèle par tâche** : Haiku pour le volume, Sonnet pour le quotidien, Opus pour la conception ; mesurer si Sonnet suffit pour rédiger le cours.
4. **Mise en cache du contexte de formation** : sur Opus 5.5, une lecture en cache coûte 0,20 $ par million de tokens au lieu de 4 $.
5. **Traitements groupés différés (batch)** : 50 % moins cher, pour les banques de questions, le quiz global du lendemain et les générations non urgentes.
6. **Effort de réflexion réglé par tâche** : bas pour la routine, car la réflexion est facturée comme de la sortie.

7. **Contexte réduit** : n'envoyer au tuteur que les passages utiles retrouvés par les embeddings, jamais des PDF entiers.
8. **Plafonds et suivi** : plafond par génération et par apprenant et par jour, avec alerte ; jetons et coût de chaque appel enregistrés, agrégés par formation.
9. **Clés personnelles** : un utilisateur qui configure son propre modèle paie sa consommation ; un modèle local peut prendre en charge les tâches légères.

Pour estimer le coût d'une formation : mesurer les jetons consommés par une formation test de bout en bout, puis appliquer les tarifs publiés de chaque modèle.

Questions ouvertes et prochaine étape

Les choix techniques sont tranchés, y compris les deux derniers points.

Sujet	Décision
Données	Supabase Cloud en région UE en V1 ; même logiciel auto-hébergeable plus tard, en migrant la base et les fichiers
Jobs	Inngest
Modèles d'IA	Configurables par chaque utilisateur, tâche par tâche, avec sa propre clé ; Claude par défaut
Validation	Prototype sur la propre formation du concepteur, SAP S/4HANA Finance : rétention FSRS par notion et coût IA réel mesurés
API	Intégrée à Next.js : API REST versionnée /api/v1, logique métier dans des paquets indépendants, extractible plus tard
Authentification	Supabase Auth : lien magique par e-mail, ou e-mail et mot de passe
Hors ligne	Application web installable avec service worker ; quiz global et cartes de rappel du jour gardés dans le navigateur (IndexedDB) ; réponses envoyées au retour du réseau, FSRS recalculé côté serveur

Sujet	Décision
Apports	Pages web : extraction du contenu principal. YouTube : sous-titres avec minutage. Images : lues par le modèle (vision). Chaque passage garde sa source pour les citations
Formations à plusieurs	Tables groupe et membres ; la Row Level Security partage programme, agenda, progression et série entre membres, et garde apports, cours, notes et lacunes par utilisateur
Paiement	Seule l'IA fournie est facturée, via un prestataire de paiement en ligne (par exemple Stripe) ; aucun paiement pour qui utilise sa propre clé

Derniers points tranchés

#	Question	Décision
1	Qui paie l'IA quand l'utilisateur garde le modèle par défaut ?	L'utilisateur : l'outil est gratuit avec sa propre clé, l'IA fournie est payante. Au plafond quotidien, bascule sur un modèle moins cher jusqu'au lendemain
2	Que faire si le modèle choisi par un utilisateur produit des briques invalides ?	La tâche s'arrête et l'utilisateur est prévenu ; il choisit un autre modèle de sa clé ou l'IA fournie, payante

Prochaine étape : un prototype technique sur un domaine test, limité au parcours d'un module. Il couvre l'import de PDF, le cours généré avec citations, le quiz de validation, les lacunes et FSRS. Il sert à mesurer la qualité, la latence et le coût réels.